

Relative path: profiling/baseline_model.py

Filename: baseline_model.py Extension: .py

```
"""
OASIS v2 – Layer 0: Baseline Profiling Model
=====
Per-entity "normal" behavior representation, per ARCHITECTURE.md Section 2.

FIX APPLIED: impossible_travel was being missed entirely because the only
geo signal (`is_new_geo`) was folded into a 9-feature average, which diluted
a single strong deviation into noise. `_geo_velocity_flag()` below is a
dedicated, non-diluted signal computed the same way cold-start rules are –
kept OUT of the averaged feature vector on purpose.

The impossibility check itself now uses geo distance / implied speed rather
than a flat time threshold.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Dict, List, Optional

import numpy as np

FEATURE_NAMES = [
    "hour_of_day",
    "session_duration_sec",
    "num_resources_touched",
    "num_new_resources",
    "auth_failures",
    "num_distinct_geo",
    "is_new_geo",
    "is_new_device",
    "sequence_length",
]

GEO_COORDS = {
    "Bengaluru,IN": (12.9716, 77.5946),
    "Mumbai,IN": (19.0760, 72.8777),
    "Singapore,SG": (1.3521, 103.8198),
    "Frankfurt,DE": (50.1109, 8.6821),
    "Ashburn,US": (39.0438, -77.4874),
    "Tokyo,JP": (35.6762, 139.6503),
    "Lagos,NG": (6.5244, 3.3792),
    "Sao_Paulo,BR": (-23.5505, -46.6333),
}

# Fastest plausible commercial flight speed, with buffer.
MAX_PLAUSIBLE_SPEED_KMH = 1000.0
SNAPSHOT_INTERVAL_DAYS = 3

def haversine_km(geo_a: str, geo_b: str) -> float:
    lat1, lon1 = GEO_COORDS[geo_a]
    lat2, lon2 = GEO_COORDS[geo_b]
    radius_km = 6371.0
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlamba = math.radians(lon2 - lon1)
    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlamba / 2) ** 2
    return 2 * radius_km * math.asin(math.sqrt(a))

def extract_features(session: dict, known_resources: set, known_geos: set = frozenset()) -> np.ndarray:
    dt = datetime.fromisoformat(session["start_time"])
    hour = dt.hour + dt.minute / 60.0
    resources = set(session["resources_touched"])
    num_new = len(resources - known_resources)
    geos = set(session["geo_locations"])
    is_new_geo = 1.0 if known_geos and not (geos & known_geos) else 0.0
    return np.array([
        hour,
        session["session_duration_sec"],
        len(session["resources_touched"]),
    ])
```

```

        num_new,
        session["auth_failures"],
        len(geos),
        is_new_geo,
        1.0 if session["is_new_device"] else 0.0,
        len(session["command_sequence"]),
    ], dtype=float)

```

```
@dataclass
```

```
class _EntityProfile:
```

```

    entity_type: str
    n_seen: int = 0
    mean: np.ndarray = field(default_factory=lambda: np.zeros(len(FEATURE_NAMES)))
    var: np.ndarray = field(default_factory=lambda: np.ones(len(FEATURE_NAMES)))
    known_resources: set = field(default_factory=set)
    known_geos: set = field(default_factory=set)
    last_geo: Optional[str] = None
    last_session_end: Optional[str] = None
    snapshot_mean: Optional[np.ndarray] = None
    snapshot_std: Optional[np.ndarray] = None
    snapshot_taken_at: Optional[str] = None

```

```
class BaselineProfiler:
```

```

    def __init__(self, cold_start_min_sessions: int = 5,
                 decay_half_life_sessions: int = 8,
                 min_std: float = 0.5):
        self.cold_start_min_sessions = cold_start_min_sessions
        self.min_std = min_std
        self.alpha = 1 - 0.5 ** (1 / decay_half_life_sessions)
        self.profiles: Dict[str, _EntityProfile] = {}
        self.population: Dict[str, _EntityProfile] = {}

```

```

    def _get_or_init(self, store: dict, key: str, entity_type: str) -> _EntityProfile:
        if key not in store:
            store[key] = _EntityProfile(entity_type=entity_type)
        return store[key]

```

```

    def _effective_mean_std(self, prof: _EntityProfile, pop: _EntityProfile):
        own_std = np.sqrt(np.maximum(prof.var, self.min_std ** 2))
        if pop.n_seen == 0:
            return prof.mean, own_std
        pop_std = np.sqrt(np.maximum(pop.var, self.min_std ** 2))
        w = min(1.0, prof.n_seen / self.cold_start_min_sessions)
        mean = w * prof.mean + (1 - w) * pop.mean
        std = w * own_std + (1 - w) * pop_std
        return mean, std

```

```

    def _geo_velocity_flag(self, prof: _EntityProfile, session: dict) -> bool:
        """Dedicated impossible-travel signal, deliberately NOT part of the
        9-feature mean (that's what caused the original miss)."""
        if prof.last_geo is None or prof.last_session_end is None:
            return False
        current_geo = session["geo_locations"][0] if session["geo_locations"] else None
        if current_geo is None or current_geo == prof.last_geo:
            return False
        if prof.last_geo not in GEO_COORDS or current_geo not in GEO_COORDS:
            return False
        try:
            last_end = datetime.fromisoformat(prof.last_session_end)
            this_start = datetime.fromisoformat(session["start_time"])
        except ValueError:
            return False
        gap_hours = (this_start - last_end).total_seconds() / 3600.0
        if gap_hours <= 0:
            return True
        distance_km = haversine_km(prof.last_geo, current_geo)
        return (distance_km / gap_hours) > MAX_PLAUSIBLE_SPEED_KMH

```

```

    def _maybe_snapshot(self, prof: _EntityProfile, session_time: datetime):
        if prof.n_seen < 3:
            return
        if prof.snapshot_taken_at is None:
            prof.snapshot_mean = prof.mean.copy()
            prof.snapshot_std = np.sqrt(np.maximum(prof.var, self.min_std ** 2)).copy()

```

```

        prof.snapshot_taken_at = session_time.isoformat()
        return
    try:
        last_snapshot = datetime.fromisoformat(prof.snapshot_taken_at)
    except ValueError:
        last_snapshot = None
    if last_snapshot is None or (session_time - last_snapshot) >= timedelta(days=SNAPSHOT_INTERVAL_DAY
S):
        prof.snapshot_mean = prof.mean.copy()
        prof.snapshot_std = np.sqrt(np.maximum(prof.var, self.min_std ** 2)).copy()
        prof.snapshot_taken_at = session_time.isoformat()

def _snapshot_drift_score(self, prof: _EntityProfile, x: np.ndarray) -> float:
    if prof.snapshot_mean is None or prof.snapshot_std is None:
        return 0.0
    idx_hour = FEATURE_NAMES.index("hour_of_day")
    idx_resources = FEATURE_NAMES.index("num_resources_touched")
    idx_new_resources = FEATURE_NAMES.index("num_new_resources")
    snapshot_std = np.maximum(prof.snapshot_std, self.min_std)
    resource_growth = max(0.0, x[idx_resources] - prof.snapshot_mean[idx_resources]) / snapshot_std[idx
x_resources]
    new_resource_growth = max(0.0, x[idx_new_resources] - prof.snapshot_mean[idx_new_resources]) / sna
pshot_std[idx_new_resources]
    hour_drift = abs(x[idx_hour] - prof.snapshot_mean[idx_hour]) / snapshot_std[idx_hour]
    score = (0.55 * resource_growth + 0.25 * new_resource_growth + 0.20 * hour_drift) / 6.0
    return float(np.clip(score, 0.0, 1.0))

def score(self, session: dict):
    """Returns (rarity_score, is_cold_start, deviation_order, geo_velocity_flag, drift_score)."""
    entity_id = session["entity_id"]
    entity_type = session["entity_type"]
    prof = self._get_or_init(self.profiles, entity_id, entity_type)
    pop = self._get_or_init(self.population, entity_type, entity_type)
    session_time = datetime.fromisoformat(session["start_time"])

    is_cold_start = prof.n_seen < self.cold_start_min_sessions
    x = extract_features(session, prof.known_resources, prof.known_geos)
    geo_velocity_flag = self._geo_velocity_flag(prof, session)
    self._maybe_snapshot(prof, session_time)
    drift_score = self._snapshot_drift_score(prof, x)

    mean, std = self._effective_mean_std(prof, pop)
    z = (x - mean) / std
    z_abs = np.abs(z)
    rarity_score = float(np.clip(1 - math.exp(-np.mean(z_abs) / 3.0), 0.0, 1.0))

    deviation_order = [FEATURE_NAMES[i] for i in np.argsort(-z_abs)]

    self._update(prof, x)
    self._update(pop, x)
    prof.known_resources |= set(session["resources_touched"])
    prof.known_geos |= set(session["geo_locations"])
    if session["geo_locations"]:
        prof.last_geo = session["geo_locations"][0]
    prof.last_session_end = session["end_time"]

    return rarity_score, is_cold_start, deviation_order, geo_velocity_flag, drift_score

def _update(self, prof: _EntityProfile, x: np.ndarray):
    if prof.n_seen == 0:
        prof.mean = x.copy()
        prof.var = np.ones(len(FEATURE_NAMES)) * (self.min_std ** 2)
    else:
        diff = x - prof.mean
        prof.mean = prof.mean + self.alpha * diff
        prof.var = (1 - self.alpha) * (prof.var + self.alpha * diff ** 2)
    prof.n_seen += 1

def entity_snapshot(self, entity_id: str) -> dict:
    if entity_id not in self.profiles:
        return {}
    p = self.profiles[entity_id]
    return {
        "entity_id": entity_id,
        "n_sessions_seen": p.n_seen,
        "mean": dict(zip(FEATURE_NAMES, np.round(p.mean, 2).tolist())),
    }

```

```
    "std": dict(zip(FEATURE_NAMES, np.round(np.sqrt(p.var), 2).tolist())),  
}
```